

Course Syllabus

Design and Analysis of Algorithms

Course Information

Course Title:	Design and Analysis of Algorithms
Course Code:	IT XXX (To be assigned)
Credits:	4 (3-0-2-0)
Lecture Hours:	3 Hours / Week
Laboratory Hours:	2 Hours / Week
Duration:	13 Weeks
Instructors:	Section A (Navjot Singh), Section B (KP Singh), Section C (Naveen Saini) Programming in C / C++ / Java / Python

Course Description

This course introduces fundamental techniques for the design and analysis of algorithms. Emphasis is placed on mathematical rigor, correctness proofs, complexity analysis, and efficient implementation. Topics include divide-and-conquer, greedy methods, dynamic programming, graph algorithms, randomized algorithms, and NP-completeness. The course prepares students for advanced study, research, and professional practice in algorithmic problem solving.

Course Outcomes

Upon successful completion of this course, students will be able to:

1. Analyze time and space complexity using asymptotic notation.
2. Design efficient algorithms using standard paradigms.
3. Prove correctness and optimality of algorithms.
4. Apply algorithmic techniques to real-world problems.
5. Implement and evaluate algorithms using appropriate data structures.
6. Understand computational hardness and approximation strategies.

Detailed Weekly Course Schedule

Week 1: Algorithmic Foundations

Lecture Topics: Introduction to algorithms and problem solving; mathematical preliminaries; growth of functions; asymptotic notation (O , Ω , Θ); worst-case and average-case analysis.

Laboratory: Empirical analysis of algorithm performance; time complexity measurement of simple algorithms.

Week 2: Divide and Conquer Paradigm

Lecture Topics: Divide-and-conquer strategy; recurrence relations; Master Theorem; binary search; merge sort and quick sort.

Laboratory: Implementation of merge sort and quick sort; experimental validation of recurrences.

Week 3: Sorting and Order Statistics

Lecture Topics: Lower bounds for comparison-based sorting; heap sort; selection problem; randomized and deterministic selection; median of medians algorithm.

Laboratory: Heap implementation; kth smallest element problems.

Week 4: Data Structures and Amortized Analysis

Lecture Topics: Review of fundamental data structures; binary heaps and priority queues; amortized analysis using aggregate, accounting, and potential methods.

Laboratory: Dynamic array implementation; amortized cost experiments.

Week 5: Greedy Algorithms

Lecture Topics: Greedy-choice property and optimal substructure; proof techniques; interval scheduling; Huffman coding; minimum spanning trees (Kruskal and Prim).

Laboratory: Union-Find implementation; MST algorithms; Huffman coding.

Week 6: Dynamic Programming I

Lecture Topics: Dynamic programming principles; overlapping subproblems and optimal substructure; 0/1 knapsack; unbounded knapsack; matrix chain multiplication.

Laboratory: Knapsack implementations; table-based and memoized DP.

Week 7: Dynamic Programming II

Lecture Topics: Longest common subsequence; edit distance; coin change problem; DP optimization techniques.

Laboratory: Space-optimized DP; sequence alignment problems.

Week 8: Graph Algorithms I

Lecture Topics: Graph representations; breadth-first search; depth-first search; topological sorting; applications of graph traversal.

Laboratory: Graph traversal implementations; cycle detection; DAG scheduling problems.

Week 9: Graph Algorithms II

Lecture Topics: Single-source shortest paths; Dijkstra's algorithm; Bellman-Ford algorithm; all-pairs shortest paths (Floyd-Warshall).

Laboratory: Shortest path implementations; negative cycle detection.

Week 10: Advanced Graph Algorithms

Lecture Topics: Correctness proofs for MST algorithms; maximum flow problem; Ford-Fulkerson and Edmonds-Karp algorithms; bipartite matching.

Laboratory: Maximum flow implementation; flow-based applications.

Week 11: Randomized Algorithms

Lecture Topics: Randomized quick sort; randomized selection; hashing and hash functions; Monte Carlo and Las Vegas algorithms; expected-time analysis.

Laboratory: Randomized algorithm implementation; hash collision analysis.

Week 12: NP-Completeness

Lecture Topics: Decision problems and complexity classes; P, NP, NP-Complete, NP-Hard; polynomial-time reductions; SAT, clique, vertex cover, and subset sum.

Laboratory: Reduction practice; backtracking and heuristic approaches.

Week 13: Approximation Algorithms and Advanced Topics

Lecture Topics: Approximation algorithms; vertex cover approximation; introduction to on-line and streaming algorithms; course review and research directions.

Laboratory: Approximation algorithm implementation.

Assessment Scheme

Assignments (Theory + Programming)	15%
Laboratory Work and Quizzes	15%
Attendance	5%
Mid-Semester Examination	25%
End-Semester Examination	40%

Recommended Textbooks

1. T. H. Cormen et al., *Introduction to Algorithms*, MIT Press.
2. J. Kleinberg and É. Tardos, *Algorithm Design*, Pearson.
3. E. Horowitz, S. Sahni and S. Rajasekaram, *Fundamentals of Computer Algorithms*, University Press.

Reference Materials

- Dasgupta, Papadimitriou, Vazirani – *Algorithms*
- Stanford CS161 Lecture Notes
- MIT 6.006 / 6.046 Course Materials